Scott Domains and Continuous Function Spaces

Small compact bases and an independent logical relation

Abstract

We give a constructive proof that the Scott-continuous functions between two Scott domains form a Scott domain. Directed and bounded suprema are computed pointwise, and the compact functions are precisely the bounded finite joins of compact step functions. The construction uses neither decidability of the order nor a choice principle. We keep the size of the order and of the compact basis explicit, explain why compatibility of pairs alone is insufficient, and prove the least-fixed-point theorem. A direct finite-stabilisation argument relates the projector's Kleene fixed point to its rank stages. On the Scott filter domain, a compatible finite-depth construction then gives an independent logical relation. We prove its typed laws and fundamental lemma, including dependent products, annotated abstraction and function extensionality, and derive mere syntactic product inversion. A short final section records the checked results and the total checking time.

1 Scott domains and small bases

We reason constructively. Fix a universe $\mathcal{U}$ of *small* types and a larger universe $\mathcal{U}^+$. Carriers of domains may belong to $\mathcal{U}^+$, whereas orders are small proposition-valued relations. Unless otherwise stated, all families whose suprema are considered have small index types. Existential assertions express mere existence: they do not include a chosen witness. These conventions are relevant to the proof; in a classical set-theoretic reading one may simply fix a size bound throughout.

Definition 1.1. Let D be a partially ordered set. An indexed family $(x_i)_{i\in I}$ is *directed* if I is inhabited and, for every $i, j \in I$, there exists $k \in I$ such that $x_i \leq x_k$ and $x_j \leq x_k$. It is *bounded* if there exists $u \in D$ such that $x_i \leq u$ for every $i \in I$.

The order is *directed complete* if every small directed family has a supremum, and *bounded complete* if every small bounded family has a supremum. An element $a \in D$ is *compact* if, for every small directed family,

$$a \leq \bigvee_{i\in I} x_i \quad\Longrightarrow\quad \exists i \in I\ a \leq x_i.$$

Definition 1.2. A *Scott domain with a small compact basis* consists of a partially ordered set D, a least element $\bot_D$, directed and bounded completeness, and a small indexed family of compact elements

$$\beta_D : \mathsf{B}_D \longrightarrow D$$

such that, for every $x \in D$, the family

$$\mathsf{B}_D(x) = \{a \in \mathsf{B}_D \mid \beta_D(a) \leq x\}$$

is directed and has supremum x.

The basis need not be injectively indexed. We normally write a for $\beta_D(a)$ when an index $a \in \mathsf{B}_D$ is used as a domain element. The least-upper-bound part of the basis condition can equivalently be written

$$\big(\forall a \in \mathsf{B}_D\ (a \le x \implies a \le y)\big) \quad \implies \quad x \le y. \tag{1}$$

Our completeness convention is for small indexed families, not arbitrary families indexed by types in $\mathcal{U}^+$.

For comparison of domains, we also use an *unpresented* form of this definition: the small compact basis is merely known to exist, rather than selected as part of the object. A presentation supplies such an object by propositional truncation.

Proposition 1.3 (Order isomorphisms give equal domain objects)**.** *Let D and E be unpresented Scott domains. An order isomorphism between their carriers induces an equality $D = E$.*

Proof. Suprema are unique, so completeness contributes no choices for a fixed order. Least elements are unique, and the basis-existence condition is a proposition. It therefore suffices to compare the orders. Univalence identifies the carrier types along the underlying equivalence. On the common carrier, the order-isomorphism laws give logically equivalent order propositions; propositional univalence makes these relations equal. The order laws themselves are propositions. The resulting equality of orders lifts to the domain objects. □

Lemma 1.4 (Finite common bounds)**.** *If finitely many compact elements $a_1, \dots, a_n$ lie below the supremum of a directed family $(x_i)_{i \in I}$, then there exists $i \in I$ with $a_r \le x_i$ for every r. In particular, if $a_1, \dots, a_n \le x$ in a Scott domain, there exists $a \in \mathsf{B}_D(x)$ above all the a_r. The statement includes $n = 0$.*

Proof. For each of the finitely many compact elements, compactness gives a family member above it. Directedness combines two such witnesses; finite induction combines all of them. For the empty family use inhabitedness of I. For the second assertion apply the first to the directed basis family with supremum x. Only finitely many existential witnesses are combined, so no choice principle is used. □

Lemma 1.5. *A bounded finite join of compact elements is compact. Every compact element of a Scott domain is equal to some basis element, in the sense of mere existence.*

Proof. Suppose $a = \bigvee_{r=1}^{n} a_r$ and $a \le \bigvee_i x_i$, where (x_i) is directed. Lemma 1.4 gives one x_i above every a_r, hence above a. This also proves compactness of the empty join $\bot$. If x is compact, apply its compactness to $x = \bigvee \mathsf{B}_D(x)$. Some $a \in \mathsf{B}_D(x)$ satisfies $x \le a$, and antisymmetry gives $x = a$. □

2 Continuous functions and pointwise suprema

Definition 2.1. A function $f : D \to E$ is *Scott-continuous* if it is monotone and, for every small directed family (x_i),

$$f\left(\bigvee_i x_i\right) = \bigvee_i f(x_i).$$

Write $[D \Rightarrow E]$ for the set of these functions, ordered pointwise.

Lemma 2.2 (A small order on functions)**.** *For $f, g \in [D \Rightarrow E]$,*

$$f \leq g \quad \iff \quad \forall a \in \mathsf{B}_D \; f(a) \leq g(a).$$

In particular the order on $[D \Rightarrow E]$ can be expressed by a small proposition, even when the carrier of D is large.

Proof. Only the reverse implication needs proof. By continuity and the basis property, for every x,

$$f(x) = \bigvee_{a \in \mathsf{B}_D(x)} f(a) \leq \bigvee_{a \in \mathsf{B}_D(x)} g(a) = g(x).$$

□

Proposition 2.3 (Pointwise suprema)**.** *The poset $[D \Rightarrow E]$ has a least element and all small directed and bounded suprema. Its least element is the constant function $\bot_E$, and these suprema are computed pointwise.*

Proof. Let $(f_i)_{i \in I}$ be a directed or bounded family of continuous maps. For every x, the corresponding family $(f_i(x))$ is directed or bounded, respectively, so define

$$h(x) = \bigvee_{i \in I} f_i(x).$$

Monotonicity of h follows from that of every f_i and the universal property of the supremum.

To prove continuity, let $(x_j)_{j \in J}$ be directed, with supremum x. Monotonicity already gives $h(x_j) \leq h(x)$. Suppose that z is any upper bound of all the $h(x_j)$. For every i, j we have $f_i(x_j) \leq h(x_j) \leq z$. Continuity of f_i gives $f_i(x) \leq z$, and therefore $h(x) \leq z$. Thus $h(x)$ is the supremum of the family $(h(x_j))$.

The pointwise universal property immediately makes h the supremum of the f_i in $[D \Rightarrow E]$. The constant bottom function is continuous and lies below every function. Taking the empty bounded supremum gives this same function. □

Lemma 2.4 (Finite input witnesses)**.** *If $f \in [D \Rightarrow E]$, $x \in D$, and b is compact in E, then*

$$b \leq f(x) \quad \Longrightarrow \quad \exists a \in \mathsf{B}_D(x) \; b \leq f(a).$$

Proof. Apply compactness of b to the directed supremum $f(x) = \bigvee_{a \in \mathsf{B}_D(x)} f(a)$. □

3 Step functions

Let a and b be compact elements of D and E, respectively. For each $x \in D$, the proposition $a \leq x$ is a small index type. The constant b family indexed by this proposition is bounded by b. We may therefore define

$$(a \Rightarrow_s b)(x) = \bigvee_{p \colon a \leq x} b. \tag{2}$$

This definition does not decide whether $a \leq x$. If $a \leq x$, its value is b; if $\neg(a \leq x)$, its value is $\bot_E$. These conditional statements do not assert a disjunction between the two cases.

Proposition 3.1. *The function $(a \Rightarrow_s b)$ is continuous and compact in $[D \Rightarrow E]$. For every continuous $f : D \to E$,*

$$(a \Rightarrow_s b) \leq f \quad \iff \quad b \leq f(a). \tag{3}$$

Proof. Monotonicity follows by enlarging the indexing proposition. To check continuity on a directed family (x_i) with supremum x, let z bound all the values $(a \Rightarrow_s b)(x_i)$. If $a \leq x$, compactness of a gives an i with $a \leq x_i$, whence $b \leq (a \Rightarrow_s b)(x_i) \leq z$. The supremum property in (2) now gives $(a \Rightarrow_s b)(x) \leq z$. The opposite inequalities follow by monotonicity.

If $(a \Rightarrow_s b) \leq f$, evaluation at a gives $b \leq f(a)$. Conversely, if $b \leq f(a)$, then $a \leq x$ implies $b \leq f(x)$; the supremum property gives $(a \Rightarrow_s b)(x) \leq f(x)$.

Finally, suppose $(a \Rightarrow_s b) \leq \bigvee_i f_i$ for a directed family of continuous maps. By (3) and pointwise computation, $b \leq \bigvee_i f_i(a)$. Compactness of b gives an i with $b \leq f_i(a)$, and then $(a \Rightarrow_s b) \leq f_i$. □

4 Bounded finite joins of steps

The function-space argument can be given directly using bounded completeness. Fix a continuous map $f : D \to E$. For compact basis elements $a_i \in \mathsf{B}_D$ and $b_i \in \mathsf{B}_E$ satisfying $b_i \leq f(a_i)$, put

$$h = \bigvee_{i=1}^{m} (a_i \Rightarrow_s b_i).$$

Each step lies below f by Proposition 3.1. Thus this finite family has the common upper bound f, and its supremum exists in the function space by pointwise bounded completeness. The empty join is the constant bottom function. There is no need to decide which entries are active at an input, or to construct a function by evaluating a graph.

Lemma 4.1. *Every bounded finite join of compact steps is compact. Moreover,*

$$\bigvee_{i=1}^{m} (a_i \Rightarrow_s b_i) \leq g \quad \Longleftrightarrow \quad \bigwedge_{i=1}^{m} b_i \leq g(a_i).$$

Proof. The characterization follows from the universal property of a supremum and the step-function characterization. For compactness, suppose the finite join lies below a directed supremum. Compactness of each step gives an index witnessing its inclusion in the directed family. Finitely many such witnesses have a common upper index, which then bounds their join. The empty case uses inhabitedness of the directed family. □

Definition 4.2. For $f : D \to E$ continuous, let $\mathcal{A}_f$ be the indexed family

$$\mathcal{A}_f = \left\{ \bigvee_{i=1}^{m} (a_i \Rightarrow_s b_i) \;\middle|\; a_i \in \mathsf{B}_D,\ b_i \in \mathsf{B}_E,\ b_i \leq f(a_i) \right\}.$$

The indices are finite lists of basis pairs with the displayed inequalities; in particular this is a small family.

Lemma 4.3. *The family $\mathcal{A}_f$ is directed and has supremum f.*

Proof. It contains the empty join. Combining two lists gives their join, still bounded by f, so it is directed.

Let h be its supremum. Clearly $h \leq f$. For the converse, fix $x \in D$ and a compact basis element $b \leq f(x)$. Continuity of f and algebraicity of D give a compact basis element $a \leq x$ with $b \leq f(a)$, by Lemma 2.4. The singleton step belongs to $\mathcal{A}_f$, and

$$b \leq (a \Rightarrow_s b)(x) \leq h(x).$$

Algebraicity of E gives $f(x) \leq h(x)$. □

5 The function-space theorem

Theorem 5.1. *If D and E are Scott domains, then $[D \Rightarrow E]$ is a Scott domain. Its compact elements are precisely the bounded finite joins of steps between compact basis elements.*

Proof. The pointwise constructions give bottom, directed completeness and bounded completeness. Lemmas 4.1 and 4.3 give algebraicity. If f is compact, apply its compactness to the directed family $\mathcal{A}_f$, whose supremum is f. Some member lies above f and also below f, hence equals f. Conversely, every member of every $\mathcal{A}_f$ is compact. □

Remark 5.2 (Small global basis). With the universe convention used here, one must additionally present all these compact functions by a single small type. This is separate from the direct approximation argument, whose indexing family $\mathcal{A}_f$ is already small for every f.

For a finite list of basis pairs, common boundedness of its steps can be expressed using only basis elements: every finite subfamily whose inputs have a common basis upper bound must have outputs with a common basis upper bound. This is a small condition. It is necessary by evaluation at a common input bound. For sufficiency, at each x take the directed supremum of the joins of output subfamilies whose inputs lie below x; unions of such subfamilies make this family directed. The resulting function is the join of the steps, by their universal property. This supplies the small global presentation. It is not needed to construct the bounded joins below a specified f in Section 4.

Remark 5.3 (Joint boundedness). For general Scott domains one must use joint boundedness of finite families. Pairwise compatibility alone is insufficient. Classically, the proper subsets of $\{1, 2, 3\}$ form a finite Scott domain under inclusion: the three singleton subsets are pairwise bounded but have no common upper bound. Taking a one-point input domain gives the corresponding counterexample to a pairwise test on steps.

6 Least fixed points

The following result requires only a pointed dcpo, although it applies in particular to every Scott domain and, by Theorem 5.1, to its continuous function spaces.

Theorem 6.1 (Kleene)**.** *Let S be a pointed dcpo and $F : S \to S$ a Scott-continuous function. Define*

$$p_0 = \bot_S, \qquad p_{n+1} = F(p_n), \qquad p = \bigvee_{n \in \mathbb{N}} p_n.$$

Then $F(p) = p$. Moreover, for every $q \in S$,

$$F(q) \leq q \quad \Longrightarrow \quad p \leq q.$$

Thus $p = \mathrm{lfp}(F)$ is the least fixed point, and is also below every pre-fixed point of F.

Proof. Bottom gives $p_0 \leq p_1$, and monotonicity of F gives inductively $p_n \leq p_{n+1}$. Hence the sequence is directed. Continuity gives

$$F(p) = \bigvee_n F(p_n) = \bigvee_n p_{n+1} = \bigvee_n p_n = p,$$

where the last equality follows since the shifted chain is cofinal. If $F(q) \leq q$, induction gives $p_0 \leq q$ and $p_{n+1} = F(p_n) \leq F(q) \leq q$. Taking the supremum proves $p \leq q$. □

In particular, after establishing continuity of a functional $\Phi : [D \Rightarrow E] \to [D \Rightarrow E]$, one can define its least solution by iteration from the constant bottom function. This construction assumes no idempotence, deflation or finitariness property of its iterates or of its limit. Any such properties must be proved separately.

7 Corrected rank stages and projector compatibility

We now apply the finite-observation method to the MIN filter domain. We use its information order, so $\bot$ is least and a consistent intersection of tokens represents their join of information. The finite tokens have bottom, universe, product, arrow and intersection forms.

Definition 7.1. Define the rank r and family-graph rank g of finite presentations by

$$\begin{aligned} r(\bot) = r(U) = g(\bot) = g(U) &= 0, \\ r(\Pi(a,f)) = g(\Pi(a,f)) &= 1 + \max(r(a), g(f)), \\ r(a \Rightarrow b) &= 1 + \max(r(a), r(b)), \\ g(a \Rightarrow b) &= \max(r(a), r(b)). \end{aligned}$$

Both ranks take maxima on intersections. Let K_n consist of the compact classes having a proper consistent representative of rank at most n. Thus $K_0 = \{\bot, U\}$. Membership is defined by the existence of a bounded representative: rank need not be invariant under changing the presentation of a compact.

Set

$$q_n(x) = \bigvee\{k \in K_n \mid k \le x\}, \qquad D_n = \{x \in D \mid q_n(x) = x\}.$$

Write $e_n : D_n \to D$ for inclusion and $r_n : D \to D_n$ for q_n with its fixed-image codomain. Each q_n is a finitary projection, the projections increase with n, and $\bigvee_n q_n(x) = x$.

7.1 A direct proof by finite unfolding

Let Φ be the continuous binary-operation functional given by the recursive projector clauses, and put

$$P_0 = \bot, \qquad P_{k+1} = \Phi(P_k), \qquad p = \bigvee_k P_k.$$

For any continuous binary operation Q, define its stage observation by

$$\rho_n(Q)(a,x) = r_n\big(Q(e_n a, e_n x)\big) \qquad (a, x \in D_n).$$

The operations P_k are not assumed to be projectors in the argument that follows.

Lemma 7.2 (One unfolding lowers the comparison stage)**.** *For all continuous binary operations Q, R,*

$$\rho_0(\Phi(Q)) = \rho_0(\Phi(R)).$$

For every n,

$$\rho_n(Q) = \rho_n(R) \quad \Longrightarrow \quad \rho_{n+1}(\Phi(Q)) = \rho_{n+1}(\Phi(R)).$$

Proof. At stage zero only the universe head can be observed, besides bottom. The universe head of $\Phi(Q)(A,X)$ is present exactly when both A and X have a universe head; this does not depend on Q.

For the successor case, use the structural equations for the rank projection. On a product it projects the domain with q_n and the family function by $g \mapsto q_n \circ g \circ q_n$. On the function component it uses the same function projection, and it preserves the universe head.

If A, X belong to stage $n+1$, the domain component of either product head belongs to stage n. Each family function, and the application function of X, takes values in stage n and factors through q_n. Consequently, in the nested calls occurring in $\Phi(Q)$, a function applied to $Q(e_n a, e_n z)$ depends only on

$$r_n Q(e_n a, e_n z) = \rho_n(Q)(a, z).$$

Its output is itself an embedded stage-n value. The outer recursive call, after projection by r_n, is therefore also determined by $\rho_n(Q)$.

For example, the restricted function clause is determined by

$$z \longmapsto \rho_n(Q)\big(B(\rho_n(Q)(a,z)),\, F(\rho_n(Q)(a,z))\big),$$

where a is the product's stage-n domain component, B its family, and F the stage-n application function of X. The product clause has the same calculation with the constant universe family. Replacing $\rho_n(Q)$ by $\rho_n(R)$ gives equal components and hence equal stage-$(n+1)$ results. These arguments use positive head observations; no decidability of a head is required. □

Corollary 7.3 (Finite stabilisation)**.** *For $k, j \geq n+1$,*

$$\rho_n(P_k) = \rho_n(P_j).$$

Proof. Induct on n. The base is the first assertion of Lemma 7.2. At the successor, write $k = k' + 1$ and $j = j' + 1$, apply the induction hypothesis to $P_{k'}, P_{j'}$, and then apply the second assertion of the lemma. □

Define the operational stage map by the finite expression

$$\widehat{p}_n = \rho_n(P_{n+1}).$$

This definition mentions neither the infinite limit p nor compact typing.

Theorem 7.4 (Operational version of Proposition 2.27)**.** *For every $a, x \in D_n$,*

$$r_n\big(p(e_n a, e_n x)\big) = \widehat{p}_n(a, x).$$

Proof. Continuity of r_n and pointwise computation of the Kleene supremum give

$$\rho_n(p) = \bigvee_k \rho_n(P_k).$$

The increasing family on the right is constant from index $n+1$ onwards, by Corollary 7.3, so its supremum is $\rho_n(P_{n+1}) = \widehat{p}_n$. □

This proves compatibility by the fixed-point construction and finite stabilisation. It uses no sandwich lemma, compact typing, or identification of p with a typing projector. The assertion that $\widehat{p}_n(a, -)$ is a finitary projector is a further property; it does not follow from stabilisation alone.

The separate identification with the projectors defined by bounded compact typing is given in Appendix A. The logical-relation construction and fundamental lemma below do not use that comparison or its rank-interpolation argument.

8 Admissible predicates and the relation clauses

We now use the Scott filter domain to relate denotations to syntax. Write $\mathbf{u} \in D$ for the universe element, $\pi(a, \varphi)$ for the product constructor, and $f \cdot x$ for application. A family φ is a function component of D, representing a continuous map $D \to D$. The recursive domain equation supplies universe observations and product observations. A positive product observation

$$\pi(\bot, \bot) \leq x$$

allows extraction of a domain component x_{d} and a family component x_{f}. These components are only used under that positive hypothesis. No decision whether an arbitrary element has a head is needed. The product $\pi(\bot, \bot)$ is a positive compact observation, distinct from $\bot$.

Fix a syntactic context Δ. Terms have variables, a universe U, dependent products $\Pi(A, B)$, annotated abstractions $\lambda A.M$, and applications. The body of a product or abstraction has one additional variable; $B[N]$ denotes substitution for this variable. Write $T \longrightarrow^* V$ for head reduction and $\Delta \vdash M = N : A$ for typed definitional equality. When such a judgement occurs inside $\| \cdot \|$, it denotes the type of its derivations, and the brackets denote propositional truncation.

8.1 Lower admissible predicates

Definition 8.1. A small proposition-valued predicate Q on D is *lower admissible* if it is downward closed and closed under small directed suprema. It is *strict* if $Q(\bot)$ holds. Strictness is a separate condition: directed families are inhabited, so admissibility alone does not imply it.

Lemma 8.2 (Compact reconstruction)**.** *For every lower admissible predicate Q,*

$$Q(x) \quad \Longleftrightarrow \quad \forall k \in \mathsf{B}_D \; (k \leq x \Longrightarrow Q(k)).$$

Conversely, for a small predicate R on the compact basis,

$$\overline{R}(x) = \forall k \in \mathsf{B}_D \; (k \leq x \Longrightarrow R(k))$$

is lower admissible. If R respects the basis order, then $\overline{R}(k) \Longleftrightarrow R(k)$ for every basis element k.

Proof. Downward closure proves one direction of reconstruction; the directed basis of x and admissibility prove the other. For admissibility of $\overline{R}$, a compact observation below a directed supremum lies below one member. Its mere witness may be eliminated into the proposition $R(k)$. Recovery on a compact uses downward closure and the observation of that compact itself. □

For lower admissible predicates P, Q, define preservation by

$$\mathrm{Arr}(P, Q)(f) \quad \Longleftrightarrow \quad \forall x \in D \; (P(x) \Longrightarrow Q(f \cdot x)).$$

Although this formula quantifies over a potentially large domain, it is equivalent to the small formula

$$\forall k \in \mathsf{B}_D \; (P(k) \Longrightarrow Q(f \cdot k)). \tag{4}$$

Indeed, if $P(x)$, then $P(k)$ for every compact $k \leq x$; continuity of application and admissibility of Q give the conclusion at x. For each compact input, application is continuous in f. Thus (4) is an intersection of lower admissible predicates of f, and is itself lower admissible. It is contravariant in P and covariant in Q.

8.2 Type and term predicates

We seek predicates

$$\mathsf{Ty}(x; T, T'), \qquad \mathsf{Elem}(x; T, M, M')$$

for all syntactic indices in Δ. Each is to be small, strict and lower admissible in x. There is no separate semantic classifier argument in Elem: semantic typing will enter its laws and the fundamental lemma.

For candidate term predicates P and Q, define the function condition $\mathcal{F}(P,Q)(f;T,M,M')$ as follows. For every reduction $T \longrightarrow^* \Pi(A,B)$, every derivation $\Delta \vdash N = N' : A$, and every x with $P(x;A,N,N')$, require

$$\begin{aligned} &Q(f\cdot x; B[N], MN, MN'),\\ &Q(f\cdot x; B[N], M'N, M'N'),\\ &Q(f\cdot x; B[N], MN, M'N'). \end{aligned} \tag{5}$$

All three comparisons use the same left result type $B[N]$. For candidate term predicates P and type predicates Q, the family condition $\mathcal{G}(P,Q)(\varphi;A,B,B')$ similarly requires

$$\begin{aligned} &Q(\varphi\cdot x; B[N], B[N']),\\ &Q(\varphi\cdot x; B'[N], B'[N']),\\ &Q(\varphi\cdot x; B[N], B'[N']) \end{aligned} \tag{6}$$

whenever $\Delta \vdash N = N' : A$ and $P(x;A,N,N')$. The compact-input formulation makes both conditions small and lower admissible in their semantic argument.

The type condition $\mathcal{T}(P,Q)(x;T,T')$ has two parts. First, a universe observation $\mathbf{u} \le x$ implies

$$\left\| T \longrightarrow^* U \;\times\; T' \longrightarrow^* U \right\|.$$

Second, a positive product observation implies the mere existence of A, A', B, B' with

$$\begin{aligned} &T \longrightarrow^* \Pi(A,B), \qquad T' \longrightarrow^* \Pi(A',B'),\\ &\Delta \vdash A = A' : U, \qquad \Delta, A \vdash B = B' : U,\\ &Q(x_{\mathrm{d}}; A, A'), \qquad \mathcal{G}(P,Q)(x_{\mathrm{f}}; A, B, B'). \end{aligned} \tag{7}$$

The whole tuple in (7) is truncated. In particular, its component equalities are retained even at the smallest positive product observation.

Lemma 8.3 (Admissibility of the type condition)**.** *The condition $\mathcal{T}(P,Q)$ is lower admissible whenever the candidate predicates are lower admissible. It is contravariant in P and covariant in Q. It is strict when the positive predicates are strict.*

Proof. Head determinism says that any two evidenced product reducts of the same pair T, T' have identical syntactic components. Consequently the product condition can equivalently be expressed by a mere existence of evidenced heads, together with the component conditions for every such evidenced head. This separates head existence from universal component tests.

Each head-existence requirement is an observation implication $k \le x \Longrightarrow E$, for a fixed compact head k and a small proposition E. Such an implication is lower admissible by compactness. The domain component can be tested by observations $\pi(k,\bot) \le x$; the family component by observations $\pi(\bot,k) \le x$. Compact reconstruction identifies these tests with the required predicates on the extracted components. Intersections give admissibility of the whole condition. Variance follows from that of argument preservation. Bottom has neither a universe head nor a positive product head, and the remaining strict output conditions are inherited from Q. □

The desired invariant equations are

$$\begin{aligned} \mathsf{Ty}(x;T,T') &\Longleftrightarrow \mathcal{T}(\mathsf{Elem},\mathsf{Ty})(x;T,T'),\\ \mathsf{Elem}(x;T,M,M') &\Longleftrightarrow \mathsf{Ty}(x;M,M') \;\wedge\; \mathcal{F}(\mathsf{Elem},\mathsf{Elem})(x;T,M,M'). \end{aligned} \tag{8}$$

These equations are mixed in variance. In particular their simultaneous diagonal action is not a monotone operator to which Kleene iteration can simply be applied.

9 Construction by compatible finite depths

Use the rank projections q_d constructed above. They satisfy $q_d \le q_{d+1} \le \mathrm{id}$, $q_d q_e = q_{\min(d,e)}$, and $\bigvee_d q_d = \mathrm{id}$. Every compact basis element is fixed by some q_d. Two pairs of predicates *agree at depth* d if their type and term components are equivalent on every element fixed by q_d, for all syntactic indices.

For a pair $R = (R^{\mathrm{ty}}, R^{\mathrm{tm}})$, form the new type predicates before the new term predicates:

$$\begin{aligned} (R^+)^{\mathrm{ty}}(x;T,T') &= \mathcal{T}(R^{\mathrm{tm}}, R^{\mathrm{ty}})(x;T,T'), \\ (R^+)^{\mathrm{tm}}(x;T,M,M') &= (R^+)^{\mathrm{ty}}(x;M,M') \ \wedge\ \mathcal{F}(R^{\mathrm{tm}}, R^{\mathrm{tm}})(x;T,M,M'). \end{aligned} \tag{9}$$

Reading the newly formed type component in the second line is essential. The original mutual action reads an old type component at the same depth and does not directly have the following one-step locality property.

Lemma 9.1 (Locality)**.** *If R and S agree at depth d, then R^+ and S^+ agree at depth $d+1$.*

Proof. For an element fixed by q_{d+1}, every extracted product domain is fixed by q_d. Its family function, and its application function, have depth-d input and output support. Write h for either such function. Then

$$h(x) = h(q_d x), \qquad q_d(h(x)) = h(x).$$

Suppose the arrow condition for R is known and an argument is related by the negative predicate of S. Lower its evidence to $q_d x$. Agreement at depth d converts this evidence to the negative predicate of R. Apply the given arrow condition at $q_d x$, use the first equation to identify the output with $h(x)$, and use the second equation to convert the positive output predicate from R to S.

This proves both directions for functions and families; the domain component is handled directly by agreement at depth d. Head evidence is unchanged. Thus the new type predicates agree at depth $d+1$. Using that agreement in the new term clause finishes the proof. □

Define the initial pair by the universe-observation tests

$$\begin{aligned} R_0^{\mathrm{ty}}(x;T,T') &= (\mathbf{u} \le x \Longrightarrow \|T \longrightarrow^* U \times T' \longrightarrow^* U\|), \\ R_0^{\mathrm{tm}}(x;T,M,M') &= R_0^{\mathrm{ty}}(x;M,M'), \qquad R_{d+1} = R_d^+. \end{aligned}$$

At depth zero, every element lies below $\mathbf{u}$, has no product head, and its applications are bottom. Hence R_0 agrees at depth zero with R^+ for every strict R. This statement does not require deciding whether a depth-zero element is bottom or the universe.

Lemma 9.2 (Compatibility and recovery)**.** *Each R_d is strict and lower admissible, and*

$$d \le e, \quad q_d x = x \quad \Longrightarrow \quad R_d(x) \Longleftrightarrow R_e(x).$$

Here and below such formulas apply separately to each component and all of its syntactic indices. Define

$$R_\infty(x) = \forall d \in \mathbb{N}\ R_d(q_d x). \tag{10}$$

Then R_∞ is strict and lower admissible, and

$$q_d x = x \quad \Longrightarrow \quad R_\infty(x) \Longleftrightarrow R_d(x).$$

Proof. The depth-zero observation and Lemma 9.1 prove compatibility by induction. Strictness and admissibility are preserved at every step. The limit is a small countable intersection of continuous pullbacks of lower admissible predicates, so has the same properties.

For recovery, the forward implication uses the dth conjunct. Conversely suppose $q_d x = x$ and $R_d(x)$. If $d \leq e$, compatibility gives $R_e(x)$, which lowers to $R_e(q_e x)$. If $e \leq d$, first lower to $R_d(q_e x)$ and then use compatibility at the depth-e element $q_e x$. Constructive comparison of natural numbers gives every conjunct. □

Theorem 9.3 (Independent invariant)**.** *There is a unique strict lower admissible pair* $(\mathsf{Ty}, \mathsf{Elem})$ *satisfying* (8)*. It is given by* (10)*.*

Proof. Recovery and locality show that R_∞ and R_∞^+ agree at every finite depth. Every compact is captured at a finite depth, so they agree on all compact observations. Lemma 8.2 gives agreement on all domain elements. The type equation then identifies the fresh type component in (9) with R_∞^{ty}, yielding both original mutual equations.

For uniqueness, a strict solution of the original equations also solves the type-before-term equations. The depth-zero observation and locality show inductively that it agrees with R_d at depth d. Recovery and compact reconstruction identify it with R_∞. □

This is a finite-depth construction with an admissible limit, rather than iteration of a monotone diagonal relation operator. It constructs the relation without an earlier logical relation or adequacy theorem. All existential eliminations have proposition-valued targets. In particular, it never replaces $\forall i\ \|W_i\|$ by $\|\forall i\, W_i\|$.

10 Typed laws and restriction of arguments

Write $p(a, x)$ for the semantic projector. Its typed-generator description and the filter soundness theorem identify semantic typing with fixedness: for a semantic type a, a value x has type a precisely when $p(a, x) = x$. In particular $p(a, x) \leq x$ is always in the image of this projector. We use the usual filter interpretation $[\![M]\!]\rho$, identifying consistent regular filters with their reflected domain elements whenever semantic typing supplies this identification.

Write $v : c$ for the inductive compact typing judgement. The global projector is generated by compact values v with $v : c$, $v \leq x$ and $c \leq a$. The following consequence is the only compact-cover property needed here; it imposes no prescribed rank bound.

Lemma 10.1 (Typed compact cover)**.** *If a compact basis element k satisfies $k \leq p(a, x)$, then there merely exist proper compact representatives v, c such that*

$$k \leq v \leq x, \qquad c \leq a, \qquad v : c.$$

Proof. The typed generators are closed under compatible finite joins and compact equivalence, and include bottom. For a join, join the classifiers as well; presupposition and conversion put both values at the common classifier, and compatible-join typing gives the resulting generator. Thus the generators below x form a directed family with supremum $p(a, x)$. Compactness of k gives a generator above it. Replacing its value and classifier by equivalent proper representatives preserves the inequalities and typing. No rank interpolation is involved. □

Lemma 10.2 (Typed relation laws)**.** *The following implications hold for all semantic a, x and syntactic indices:*

$$\begin{aligned}
\mathsf{Ty}(p(\mathbf{u},a);T,T') &\Longrightarrow \mathsf{Ty}(p(\mathbf{u},a);T',T),\\
\mathsf{Ty}(p(\mathbf{u},a);T,V) \ \wedge\ \mathsf{Ty}(p(\mathbf{u},a);V,W) &\Longrightarrow \mathsf{Ty}(p(\mathbf{u},a);T,W),\\
\mathsf{Elem}(p(a,x);T,M,N) &\Longrightarrow \mathsf{Elem}(p(a,x);T,N,M),\\
\mathsf{Elem}(p(a,x);T,M,N) \ \wedge\ \mathsf{Elem}(p(a,x);T,N,O) &\Longrightarrow \mathsf{Elem}(p(a,x);T,M,O),\\
\mathsf{Ty}(p(\mathbf{u},a);T,T') \ \wedge\ \mathsf{Elem}(p(a,x);T,M,N) &\Longrightarrow \mathsf{Elem}(p(a,x);T',M,N).
\end{aligned}$$

The relation also has head-reduction transport, expansion in both term arguments, and left diagonal closure.

Proof. Prove the five typed implications together at finite relation depth, with the type implications before the term implications at each successor. The semantic projector is eliminated into lower admissible predicates using its universe, product and function cases. In the function case its action has the form

$$p(\pi(a,\varphi),f)\cdot z = p(\varphi\cdot p(a,z),\ f\cdot p(a,z)).$$

Restrict the tested argument to $p(a,z)$. Idempotence identifies the result of testing at z with that of testing at $p(a,z)$, and puts the output in the corresponding codomain projector. The depth induction hypotheses therefore supply the required argument conversions and output symmetry or transitivity. The type clauses supply the head and component evidence needed to align the tests.

Passing to the invariant requires a typed cover, not an assumption that rank projections preserve typing. Given a compact observation below $p(a,x)$, Lemma 10.1 supplies a compact value v and classifier c with

$$k \le v \le x, \qquad c \le a, \qquad v : c.$$

Choose proper representatives and a common rank bound for v and c. The value is fixed by $p(c,-)$, and the classifier is universe-typed. Recovery transfers the premises to the finite-depth relation at this bound; the finite-depth laws apply; recovery transfers the conclusion back. Lowering to k and compact reconstruction prove the laws on the whole projected element. Type symmetry and transitivity use the same argument with the universe projector.

Head transport and term expansion follow from the clauses and head determinism. Left diagonal closure is proved first at finite depth and then passed to the limit. It does not require semantic typing. Semantic soundness transfers the five projected laws to typed filter denotations. □

10.1 Fitting environments and related substitutions

An environment fits the empty context automatically. It fits $\Gamma, z : A$ if its restriction ρ fits Γ, the denotation $[\![A]\!]\rho$ is universe-typed, and its last value is semantically typed by $[\![A]\!]\rho$. We write $\rho \models \Gamma$.

Let $\sigma, \sigma' : \Gamma \to \Delta$ be typed substitutions. Relative to a fitting environment ρ, say they are *related* if for each variable z of Γ, with its type A_z expressed in Γ, we have derivations

$$\Delta \vdash \sigma z = \sigma' z : A_z[\sigma], \qquad \Delta \vdash \sigma' z = \sigma z : A_z[\sigma'],$$

and both relations

$$\mathsf{Elem}(\rho z; A_z[\sigma], \sigma z, \sigma' z), \qquad \mathsf{Elem}(\rho z; A_z[\sigma'], \sigma z, \sigma' z). \tag{11}$$

Equivalently, every proper compact observation of each raw environment filter satisfies these relations. Typed symmetry allows reversal of related substitutions; diagonal closure allows replacing a related pair by its left diagonal. These statements hold in a well-formed source context and use only the variable typings supplied by fittingness.

Lemma 10.3 (Restriction to a fitting extension)**.** *Suppose $\rho \models \Gamma$, the substitutions σ, σ' are related, and $\Gamma \vdash A : U$. Suppose the domain induction hypothesis provides*

$$\mathsf{Ty}(a; A[\sigma], A[\sigma']), \qquad a = [\![A]\!]\rho.$$

Given an arbitrary argument test

$$\Delta \vdash N = N' : A[\sigma], \qquad \mathsf{Elem}(x; A[\sigma], N, N'),$$

put $y = p(a, x)$. Then $\rho[z \mapsto y] \models \Gamma, z : A$, and $(\sigma, z \mapsto N)$ and $(\sigma', z \mapsto N')$ are related for that extended environment.

Proof. The raw semantic projection representing y is typed by a, since a is universe-typed. Thus the extended environment fits. Since $y \leq x$, downward closure gives the argument relation at $A[\sigma]$. The conversion law of Lemma 10.2, together with the type relation at a, gives it at $A[\sigma']$. More explicitly, the type premise lowers to $p(\mathbf{u}, a)$, and idempotence identifies $p(a, y)$ with y.

Syntactic cross-substitution applied to $A : U$ gives $\Delta \vdash A[\sigma] = A[\sigma'] : U$. Symmetry and conversion of the argument equality give the required equality in the other direction at $A[\sigma']$. The old variables retain their relations, since substitution in a weakened type ignores the newly substituted argument. These are exactly the conditions for the two extended substitutions. □

10.2 Why restricting the argument suffices

For a body M over $\Gamma, z : A$, let $\Lambda_{A,M,\rho}$ denote the semantic binder filter: its finite graph entries have a compact input u typed by $[\![A]\!]\rho$ and an output observed in $[\![M]\!](\rho[z \mapsto u])$. This is the semantic abstraction used both for lambda bodies and for product families.

Lemma 10.4 (Evaluation through the restriction)**.** *Let $a = [\![A]\!]\rho$ and $y = p(a, x)$. Every observation of $\Lambda_{A,M,\rho} \cdot x$ is an observation of $[\![M]\!](\rho[z \mapsto y])$. Hence, whenever the latter is a typed denotation,*

$$\Lambda_{A,M,\rho} \cdot x \leq [\![M]\!](\rho[z \mapsto y]). \tag{12}$$

The same implication holds for any function component below the binder.

Proof. For a compact input presentation u, consider the auxiliary family

$$H(u) = [\![M]\!](\rho[z \mapsto p(a, u)]).$$

It is monotone in the information order. At a graph entry whose input u is typed by a, the principal filter generated by u is fixed by $p(a, -)$. Thus that entry's output belongs to $H(u)$.

An application observation selects finitely many entries of a binder graph under an input observed in x. Induction on the selection, using monotonicity for a single entry and filter intersection for a combined selection, puts the selected output in $H(u_0)$ at that observed input u_0. Since $u_0 \leq x$, projector monotonicity gives $p(a, u_0) \leq p(a, x)$, so the output belongs to the right-hand side of (12). Upward closure in token order handles the final observation. The argument is on raw filter witnesses; after reflection, truncated witnesses are eliminated only into the desired proposition. Compact reconstruction then transfers any lower admissible predicate of the body denotation to the application result. □

There is a second point in the function and family clauses: the induction hypothesis must supply three comparisons, all at the same left type. Use the fitting extension from Lemma 10.3 and write

$$V = B[\sigma, N], \quad Y = [\![M]\!](\rho[z \mapsto y]),$$
$$P = M[\sigma, N], \quad P' = M[\sigma, N'], \quad Q = M'[\sigma', N], \quad Q' = M'[\sigma', N'].$$

The notation $M[\sigma, N]$ means substitution by σ on the old variables and by N on the new one. A self induction hypothesis for M at the diagonal base substitution yields $\mathsf{Elem}(Y; V, P, P')$. The cross induction hypothesis for M, M' yields

$$\mathsf{Elem}(Y; V, P, Q'), \qquad \mathsf{Elem}(Y; V, P, Q),$$

the second by the diagonal argument test (N, N). Typed symmetry and transitivity on Y now give

$$\mathsf{Elem}(Y; V, P, P'), \qquad \mathsf{Elem}(Y; V, Q, Q'), \qquad \mathsf{Elem}(Y; V, P, Q'). \tag{13}$$

This obtains the right self comparison without changing its classifier to $B[\sigma', N]$ and without choosing a uniform family of witnesses.

11 The fundamental lemma and product inversion

Theorem 11.1 (Fundamental lemma)**.** *Suppose Δ is well formed, $\rho \models \Gamma$, and $\sigma, \sigma' : \Gamma \to \Delta$ are related relative to ρ. For every derivation $\Gamma \vdash M = N : A$,*

$$\begin{aligned} &\mathsf{Elem}([\![M]\!]\rho; A[\sigma], M[\sigma], N[\sigma']), \\ &\mathsf{Elem}([\![M]\!]\rho; A[\sigma'], M[\sigma], N[\sigma']). \end{aligned} \tag{14}$$

For a typing derivation $\Gamma \vdash M : A$, the same conclusions hold with $N = M$. They hold on the whole reflected denotation and on every compact observation below it.

Proof. Induct simultaneously on typing and equality derivations, with the well-formed-context cases contributing no relational assertion. The hypotheses are quantified over all fitting environments and related substitutions. We give the cases that explain the construction.

Variables, universe, and structural rules. The variable case is (11). The universe case uses its own head reduction and the absence of a product reduct of U. Reflexivity uses the typing hypothesis. Symmetry reverses the related substitutions, applies the induction hypothesis, and uses typed symmetry; semantic soundness identifies the denotations of the equal terms. Transitivity uses the diagonal substitutions and typed transitivity. Conversion applies the type induction hypothesis and the conversion law in Lemma 10.2, at each substituted classifier.

Dependent products. For formation, the premises are $\Gamma \vdash A : U$ and $\Gamma, z : A \vdash B : U$. For equality they additionally include $\Gamma \vdash A = A' : U$ and $\Gamma, z : A \vdash B = B' : U$, with the specified typings of the bodies. Substitution and cross-substitution give the syntactic component equalities

$$\Delta \vdash A[\sigma] = A'[\sigma'] : U, \qquad \Delta, z : A[\sigma] \vdash B[\sigma^+] = B'[\sigma'^+] : U.$$

Here σ^+ denotes the lifted substitution under the binder. These equalities come from the premises, not from inversion of a product equality. The domain induction hypothesis gives the type relation on $[\![A]\!]\rho$.

The product denotation has no universe head. Under a positive product observation, its extracted domain is below $[\![A]\!]\rho$, and its extracted family is below $\Lambda_{A,B,\rho}$. For an arbitrary

related argument, Lemma 10.3 gives a fitting extension. The body induction hypotheses and (13), with classifier U, supply the three type comparisons in (6). Lemma 10.4 transfers them to the extracted family. Fold the type clause, then the term clause at U. Formation is the self-equality instance. Since the classifier is U, the two substituted classifier conclusions coincide.

Annotated abstraction. For $\lambda A.M : \Pi(A, B)$, its semantic value has only a function component, so the type component of the term relation has no positive head to test. For a related argument, restrict it as above, use the body induction hypothesis and (13), and transfer the outputs by (12). Syntactic beta reduction and term expansion replace the instantiated bodies by applications of the substituted abstractions. This proves the three function outputs at $\Pi(A, B)[\sigma]$. Apply the same argument with σ, σ' reversed and then typed symmetry on the abstraction denotation to obtain the conclusion at $\Pi(A, B)[\sigma']$.

Application and beta. The function relation applied to the argument relation yields the application comparison at the left substituted result type. Instantiating the codomain induction hypothesis under the binder gives the relation between the two substituted result types; typed conversion gives the right-type conclusion. This treats ordinary application and both congruence rules. For beta, extend the environment by the argument denotation and the substitutions by the two substituted arguments. The body hypothesis applies there. Semantic beta and substitution identify the denotation, while syntactic beta and term expansion give the required equality comparison at both result types.

Typed function extensionality. Suppose the premises include

$$\Gamma, z : A \vdash f^{\uparrow} z = g^{\uparrow} z : B, \qquad \Gamma \vdash f : \Pi(A, B), \quad \Gamma \vdash g : \Pi(A, B).$$

Semantic typing gives the eta inclusions

$$[\![f]\!]\rho \leq \Lambda_{A, f^{\uparrow} z, \rho}, \qquad [\![g]\!]\rho \leq \Lambda_{A, g^{\uparrow} z, \rho}. \tag{15}$$

To see the first, take a typed compact cover of an observation of $[\![f]\!]\rho$. A non-bottom cover must be a function graph: universe and product values cannot have a product classifier. Each graph entry has a key typed by $[\![A]\!]\rho$ and its output is observed in the application of $[\![f]\!]\rho$ to that key. Semantic weakening identifies this with the corresponding eta body. All entries therefore belong to the binder, and the graph covers the original observation. The proof for g is the same. This is a consequence of semantic typing and soundness.

The pointwise equality induction hypothesis, together with its symmetry and transitivity consequences, supplies the self and cross hypotheses for the two eta bodies. Restrict each tested argument to a fitting extension, obtain (13), and use (15) and Lemma 10.4 to transfer them to applications of $[\![f]\!]\rho$. Substitution in $f^{\uparrow} z$ is exactly application of $f[\sigma]$ to the substituted argument. Hence the three outputs fold to the required function relation. For the right classifier, reverse both the pointwise equality and the substitutions, use typed symmetry, and transfer from the denotation of g to that of f by semantic soundness of the extensionality rule.

These cases cover every derivation constructor. The proof can be read observation by observation on the raw filters. Semantic typing gives consistent regular denotations, and Lemma 8.2 then supplies the whole-denotation assertions. Conversely, downward closure gives every compact-observation conclusion. □

Corollary 11.2 (Mere product inversion)**.** *From a derivation* $\Gamma \vdash \Pi(A, B) = \Pi(A', B') : T$*, for any classifier* T*, one obtains*

$$\left\| (\Gamma \vdash A = A' : U) \times (\Gamma, z : A \vdash B = B' : U) \right\|.$$

Proof. The environment $\rho_\bot$ assigning bottom to every variable fits every well-formed context. Prove this by induction on the context: soundness makes the next type universe-typed, and bottom is a value of that type. The identity substitution is related to itself at $\rho_\bot$ by strictness of the relation.

The literal product denotation has the compact observation $k_\Pi = \pi(\bot, \bot)$: use the bottom observation of its domain and the empty family graph. Apply Theorem 11.1 at the identity substitutions and observe k_Π. The type component of the term clause gives $\mathsf{Ty}(k_\Pi; \Pi(A, B), \Pi(A', B'))$. Unfolding the positive product clause supplies a mere tuple of evidenced heads with their component equalities. Literal products are head normal, so head determinism identifies those components with A, B, A', B'. Mapping this truncated tuple to its equalities gives the displayed conclusion. □

The argument derives product inversion after the fundamental lemma; it uses neither syntactic product injectivity nor an earlier adequacy theorem as a premise. The conclusion is explicitly truncated. It does not assert a chosen pair of derivation trees, or a simultaneous choice of witnesses over all compact observations.

A Comparison with bounded typing

This appendix identifies the operational stage maps with the projectors defined by bounded compact typing. Its rank-interpolation argument is independent of the logical-relation construction and fundamental lemma in the main text. Write $u : b$ for the inductive MIN compact typing judgement. It includes $\bot : \bot$, $\bot : U$ and $U : U$, and is closed under conversion and compatible joins. If $u : b$, $b \le b'$ and $b' : U$, then $u : b'$. Product and function typing use typed graph keys and outputs, with a well-formed classifier.

For bounded proper representatives define $u :_n b$ to mean $u : b$ with $r(u), r(b) \le n$. For $A \in D_n$, let $T_n(A)$ be the compact family generated by the values u for which $u :_n b$ for some $b \le A$, closed under compact equivalence and compatible finite joins. Define

$$p_n(A, X) = \bigvee \{u \in T_n(A) \mid u \le X\}.$$

Here p_n denotes the typing-defined stage projector, in distinction to the operational map $\widehat{p}_n$. This defines p_n independently of the global projector p. Its image lies in D_n, since $T_n(A) \subseteq K_n$.

Lemma A.1 (Two-sided rank interpolation)**.** *Suppose L, X, A have proper consistent presentations of rank at most n. If $u : b$ and*

$$L \le u \le X, \qquad b \le A,$$

there are proper presentations u', b' of rank at most n such that

$$u' : b', \qquad L \le u' \le X, \qquad b' \le A.$$

There is no additional requirement that $u' \le u$ or $b' \le b$.

Proof. Induct on n, and within a stage on the lower token L. Bottom is handled by $\bot : \bot$. A universe observation forces the typed value and its classifier to be U, up to equivalence. Intersections of lower observations are handled by joining the two interpolated values and their classifiers. The upper caps ensure consistency; monotonicity in the classifier and closure of typing under joins give the joined typing. These cases suffice at stage zero.

We describe the successor cases. Finite graph selection has two elementary properties. A selection from a typed graph has a typed key and a typed output. A selection from a graph whose entries have rank at most n has a key and output of rank at most n. Both assertions follow by induction on finite selections, using joins for a combined selection. When a bounded graph dominates a step $t \Rightarrow w$, a selection supplies a bounded key $\mu \le t$ and output $c \ge w$ such that $\mu \Rightarrow c$ lies below that graph.

First take a lower function observation $k \Rightarrow v$. In the non-bottom case, select from the typed function a key $t : B$, a value $w : b$ and a classifier-family observation $t \Rightarrow b$, with $t \le k$ and $v \le w$. From the upper value graph select a support $\mu_X \le t$ and an output cap $c_X \ge w$. From the upper product classifier extract its domain cap B_* and family graph, and select from that graph a support $\mu_A \le t$ and output cap $c_A \ge b$. All these caps and supports have rank at most n.

Apply the induction hypothesis to $t : B$ between $\mu_X \vee \mu_A$ and k, with classifier cap B_*. It gives $t' : B'$ in that interval. Apply it to $w : b$ between v and c_X, with classifier cap c_A, to obtain $w' : b'$. Then

$$(t' \Rightarrow w') : \Pi(B', t' \Rightarrow b')$$

is the required bounded typing. The lower bounds on t' preserve both upper graph bounds; its upper bound $t' \le k$ ensures that the new step covers the lower observation. Presupposition gives $b' : U$, so the new product classifier is well formed.

For a lower product observation $\Pi(d, h)$, generation gives a typed product with classifier U. Interpolate its domain between d and the extracted upper domain cap. For each step of h, select a typed key and a universe-typed output from the original family. Select a bounded support and output cap from the upper family, then interpolate the key and output at stage n. Join the resulting domain classifiers with the domain already obtained, and join the resulting family steps. All keys can be typed at this joined domain by monotonicity; all outputs remain typed at U. The upper domain and family caps ensure consistency. The resulting typed product has rank at most $n + 1$, covers the lower product, and remains below the upper value cap. This completes the induction. □

Theorem A.2 (Corrected rank-stage version of Proposition 2.27)**.** *For the global projector p defined by the Kleene iteration of its recursive functional, and for every $A, X \in D_n$,*

$$r_n(p(e_n A, e_n X)) = p_n(A, X).$$

Proof. Identify stage elements with their images in D. A compact observation of $q_n(p(A, X))$ is covered by some $L \in K_n$ lying below $p(A, X)$. Lemma 10.1 supplies $u : b$ with $L \leq u \leq X$ and $b \leq A$. Because $X = q_n(X)$ and $A = q_n(A)$, compactness supplies bounded compact caps X_*, A_* with $u \leq X_* \leq X$ and $b \leq A_* \leq A$. Apply Lemma A.1. Its witnesses generate $p_n(A, X)$ and cover L, proving $q_n(p(A, X)) \leq p_n(A, X)$.

Conversely every bounded typed generator is a global typed generator, and the global compact family is closed under the joins and equivalences used to form $T_n(A)$. Thus $p_n(A, X) \leq p(A, X)$. Since $p_n(A, X)$ is fixed by q_n, monotonicity of q_n proves the reverse inequality. The identification of the Kleene solution with the typed-generator projector justifies using that characterization for the independently constructed p. □

Combining this theorem with Theorem 7.4 gives $\widehat{p}_n = p_n$. In particular the operational stage maps inherit finitariness from the compact-family construction. This last comparison still uses the sandwich lemma; the operational compatibility theorem above does not.

This comparison theorem uses the corrected MIN rank presentation and its bounded typing relation. A comparison with a separately defined stage-indexed relation from the paper, or with a separately constructed successor domain functor, is a further statement.

Formal checks and total timing

The development is checked in Unit with univalence and replacement as ambient parameters. Function extensionality is derived from univalence; propositional truncation is obtained from the replacement interface. No excluded middle, choice or propositional resizing axiom is added. The main checked results are:

1. Scott domains with small compact bases, including an object with merely existing basis and equality induced by order isomorphism; continuous function spaces, compact step approximations, and least fixed points.

2. The Scott filter domain and recursive projector; corrected rank stages, finite stabilisation, and comparison with bounded compact typing.

3. The independent strict lower admissible invariant, its equations and uniqueness; typed symmetry, transitivity and conversion.

4. Typed argument restriction; Pi formation and equality, annotated lambda and function extensionality; the complete derivation induction.

5. Whole-denotation adequacy at both substituted types, compact recovery, and mere syntactic Pi inversion.

The complete induction is `At.fundamental` in `IndependentFundamental.seg`; its consequences are in `IndependentAdequacy.seg` and `IndependentInversion.seg`.

A run on 12 November 2026 of `/usr/bin/time -p python3 UNIT/check.py`, using the recorded local macOS arm64 checker, gave the following results. The type-checker is a compiled Haskell program. Python only verifies the dependency hashes and invokes that program for the two checks; `time -p` measures the complete command.

Check	Declarations	Checker CPU time
Complete development	9,414	2.31 s
Independent relation and consequences	4,909	1.10 s

The **total wall-clock time was 3.50 seconds**, including dependency verification and both checker processes. The independent run has a 104-module dependency closure with the old logical relation and the earlier proofs of adequacy, syntactic Pi injectivity and subject reduction out of scope. All 40 frozen dependency hashes were verified unchanged. The two declaration counts overlap; the full count also includes developments outside this account. These timings describe this local run, not a machine-independent bound.

The inversion conclusion remains propositionally truncated. Comparison with the old uniform proof-relevant relation and transfer to the stable reflexive domain remain separate. The corrected rank-stage theorem does not validate the unamended printed stage-zero rules, whose obstruction is checked separately, nor identify the stages with a separately encoded successor-domain functor. The classical finite-poset counterexample is illustrative and has not been formally checked.